

P2P CLI Messaging Tool

Terminal chat · multi-peer · persistent history · OS notifications · no bind errors

Python 3.11+

asyncio

Textual TUI

SQLite

UDP P2P

Quick start

Terminal 1 — rendezvous server

run once, leave running

```
python rendezvous_server.py
```

Terminal 2 — alice

first peer

```
python tui.py alice bob
```

Terminal 3 — bob

second peer (different port — same machine only)

```
python tui.py bob alice --port 9001
```

■ On separate machines, both peers use the default port — no `--port` flag needed. The flag is only required when sharing one machine.

Connect to multiple peers at once

alice talks to bob and carol simultaneously

```
python tui.py alice bob carol
```

Add a peer while the app is running

```
/add dave
```

Clone & install

Clone

```
git clone https://github.com/jatin0-0/P2P-CLI-Messaging-Tool.git
cd P2P-CLI-Messaging-Tool
```

Virtual environment (recommended)

```
python -m venv venv
```

```
# Windows
```

```
venv\Scripts\activate
```

```
# macOS / Linux
```

```
source venv/bin/activate
```

Install dependencies

```
pip install textualaiosqlite aiohttp flask plyer
```

- `plyer` is optional — OS popup notifications silently skipped if absent. `flask` is only needed on the machine running the rendezvous server.

Windows filename warning

Windows is case-insensitive but Python imports are not. Ensure filenames are lowercase or you'll get a circular import error:

```
# Rename in PowerShell if needed
Rename-Item PeerManager.py peer_manager.py
Rename-Item PeerWorker.py peer_worker.py
```

Usage

Cross-network (different Wi-Fi / mobile hotspot)

Deploy `rendezvous_server.py` on any VPS, then update `config.py`:

```
RENDEZVOUS_HOST = "YOUR_SERVER_PUBLIC_IP"
```

Both peers run `tui.py` normally. The tool auto-picks the best connection path.

In-app commands

Command	What it does
<code>/add</code>	Connect to a new peer without restarting
<code>/quit</code>	Exit cleanly, flush database
<code>Ctrl+C</code>	Also exits
<code>Ctrl+N</code>	Focus the message input box

Connection paths (fully automatic — no flags needed)

Path A — direct LAN

Same /24 subnet (e.g. both on 192.168.1.x)

Path B — UDP hole punch

Different networks, cone NAT — 3 second timeout

Path C — HTTP relay

Symmetric NAT or no public IP — routes via rendezvous server

History

All messages are stored in `chat.db` (SQLite). History loads automatically when you open a peer tab — no setup needed.

Architecture

One UDP socket, bound once by `peer_manager.py` for the entire process lifetime. All peers share it. Incoming packets are routed to the correct `PeerWorker` by the sender's (ip, port) — making bind conflicts structurally impossible.

File	Role	Description
<code>tui.py</code>	Entry point	Textual UI, tabs per peer, unread badges, OS notifications
<code>peer_manager.py</code>	Socket owner	Binds 0.0.0.0:9000 once, routes packets to workers
<code>peer_worker.py</code>	Per-peer task	asyncio task — LAN / hole punch / relay state machine
<code>storage.py</code>	Persistence	SQLite via aiosqlite — messages + peer address records
<code>stun_client.py</code>	STUN	Discovers public IP/port via RFC 5389 — unchanged
<code>rendezvous_server.py</code>	Server	Flask meeting-point — register, discover, relay
<code>config.py</code>	Constants	All tunable settings
<code>chat.db</code>	Database	Created automatically on first run — don't commit to git

Config reference

All constants live in `config.py`:

Key	Default	Description
<code>RENDEZVOUS_HOST</code>	127.0.0.1	Change to VPS IP for cross-network use
<code>RENDEZVOUS_PORT</code>	5000	Rendezvous server port
<code>LOCAL_UDP_PORT</code>	9000	Default UDP port (one per machine)
<code>DB_PATH</code>	<code>chat.db</code>	SQLite database file location
<code>KEEPALIVE_INTERVAL</code>	10	Seconds between PING packets
<code>RECONNECT_DELAY</code>	5	Seconds before retry after silence
<code>MAX_HISTORY_DISPLAY</code>	200	Messages loaded from DB on tab open
<code>LONG_POLL_TIMEOUT</code>	30	Rendezvous long-poll timeout (seconds)